

ML Specification Report — playground-series-s3e1

California Housing — Median House Value Regression · our from-scratch agent (GBDT pool + tree-search v2)

Figures grounded in the competition's `facts.json`, `eda_summary.json`, `STATUS.md`, and `experiments_tree.json`.

Overview

The task is to predict a census block's median house value (`MedHouseVal`) on the California-Housing Playground Series episode (metric: RMSE, lower is better). We solve it with a from-scratch pipeline — a pool of gradient-boosted trees (LightGBM / XGBoost / CatBoost) grown from an 8-feature baseline into a 26-feature engineered set, refined by four linear self-improvement rounds and then a 22-node tree search. Our champion scores **CV OOF RMSE 0.556329**; the NVIDIA reproduce-agent, copying a strong public coordinates-engineering kernel verbatim, reaches **0.5086** — NVIDIA ahead by a wide margin on this comp.

Why it matters. Accurate house-value modelling underpins mortgage lending, property taxation, and regional planning, and the California Housing data is a canonical benchmark for tabular regression.

Data

Purpose of Data. Predict the median house value of a California census block from 8 tabular census/geographic descriptors — a continuous-target **regression** Playground Series episode. **Data Format** is clean **tabular CSV**, all-numeric (no categorical columns), with an `id` column and the `MedHouseVal` target. **Data Volume** is **37,137 training rows / 24,759 test rows**, with 8 raw predictive features once `id` and the target are removed (`MedInc`, `HouseAge`, `AveRooms`, `AveBedrms`, `Population`, `AveOccup`, `Latitude`, `Longitude`).

Data Quality is high: **no missing values** in either split, **0 duplicate rows**, and no meaningful train ↔ test covariate shift (largest mean difference is `Population` at 1.12%, all others <0.5% — so no reweighting or adversarial validation is needed). The main modelling wrinkles are distributional rather than dirty data: the target is right-skewed (skew 0.97) and, critically, **top-coded at exactly 5.00001 for 4.92% of train rows** (`HouseAge` is likewise top-coded at 52 for 5.57% of rows) — a known California-Housing capping artifact that puts a floor on achievable RMSE regardless of modelling. Several features carry very long right tails (`AveOccup` skew 170.9, max ~503; `Population` skew 5.8; `AveBedrms` skew 13.0) driven by a handful of very-low-occupancy blocks. The strongest EDA signals were:

- **MedInc** is by far the strongest linear predictor (Pearson $r \approx +0.70$, Spearman $r \approx +0.70$); `AveRooms` is next ($r \approx +0.37$).
- **Latitude/Longitude** look weak linearly ($r \approx -0.12$ / -0.06) but a quick raw-feature LGB ranked **Longitude and Latitude as the top-2 importance features** — the geo signal is non-linear (location-

based, not distance-from-origin), and the two coordinates are strongly anti-correlated ($r \approx -0.937$) along California's NW–SE coastline.

- Among engineered features, `rooms_per_person` was the strongest new signal ($r \approx +0.452$), then `dist_nearest_city` ($r \approx -0.344$) and `log_AveRooms` ($r \approx +0.342$).

Annotation Guidelines. The label is the continuous `MedHouseVal` (median house value, in the dataset's price units, range ≈ 0.15 – 5.00001). Submissions are **real-valued predictions** scored by RMSE — there is no class label or ranking.

Feature Set. From the 8 raw columns we engineer up to **26 features** (8 raw + 16 in the main pipeline, +2 adopted in round 3), grouped below:

Group	Features
Raw	<code>MedInc</code> , <code>HouseAge</code> , <code>AveRooms</code> , <code>AveBedrms</code> , <code>Population</code> , <code>AveOccup</code> , <code>Latitude</code> , <code>Longitude</code>
Ratios	<code>households</code> (=Population/AveOccup), <code>bedroom_ratio</code> (AveBedrms/AveRooms), <code>rooms_per_person</code> (AveRooms/AveOccup)
Log1p transforms	<code>log_AveRooms</code> , <code>log_AveBedrms</code> , <code>log_Population</code> , <code>log_AveOccup</code> , <code>log_households</code> (tame heavy right tails)
Geo — distance	<code>dist_LA</code> , <code>dist_SF</code> , <code>dist_SanDiego</code> , <code>dist_Sacramento</code> , <code>dist_SanJose</code> , <code>dist_nearest_city</code> , <code>coastal_dist</code> (round 3)
Geo — spatial	<code>lat_long</code> interaction, <code>geo_cluster</code> (KMeans $k=25$, fit on train coords), <code>knn_mean_dist_10</code> (round 3, KNN $k=10$ local-density, computed leakage-safe over combined train+test coords)

Splitting strategy. A single canonical **5-fold KFold (shuffle=True, seed = 42)**, no stratification or grouping, shared across all experiments so scores are directly comparable. The justification is textbook: a continuous target with no natural strata, each row an independent census block (no group/time structure), and near-identical train/test distributions. Fold RMSE for the winning blend's members ranged ~ 0.545 – 0.588 (naturally noisier than a large-N classification task, but no fold is a systematic outlier). The leaderboard was **not** exercised this run (offline, unattended batch — no Kaggle credentials touched), so all scores here are **CV-only OOF RMSE**.

Models & Architecture

Purpose of Architecture. A regression model that minimizes RMSE on median house value. **Architecture Type** is a **gradient-boosted decision-tree ensemble** — LightGBM as the primary learner, with XGBoost and CatBoost members plus, in the tree-search champion, an Optuna-tuned LGB, a seed-bagged sibling, a regularization-nudged variant, a deliberate-diversity deep XGB, and a two-stage ceiling-classifier hybrid — combined by a weighted blend. The champion is a **7-member blend** discovered at tree-search node #14.

Each member takes the same **Input Format** — a numeric feature matrix (all engineered features are numeric; no categorical encoding is required) — of **Input Dimension 26 features** per row (a 1-D vector; there is no spatial/sequence structure fed to the model beyond the engineered geo features).

Architecture Description. Each GBDT member is an additive ensemble of decision trees grown by histogram-based gradient boosting (representative LGB: `num_leaves` = 63, `n_estimators` = 2000 with early stopping; the Optuna-tuned LGB uses `num_leaves` = 121, `max_depth` = 10). The blend layer sits on top as a convex weighted average of the members' out-of-fold predictions, with weights found by grid search (step 0.05) in the linear rounds and by Dirichlet($k=800$) sampling + coordinate-ascent in the tree search. The one non-regressor member is the **ceiling-classifier hybrid**, a two-stage model whose cap-probability classifier (OOF AUC 0.964) targets the 5.00001 top-code — it is a poor solo regressor (0.561227) but the largest-weighted, biggest-gain blend member. **Model Complexity** is best expressed as trees $\times$ leaves rather than a dense parameter count: a single LGB member $\approx 2000 \times 63$ leaves (pre-early-stopping ceiling), and the champion blends 7 such members.

Training procedures

Training proceeded in two phases on the shared 5-fold split, each measured on the same OOF RMSE scale:

Phase / exp	Configuration	OOF RMSE
exp 1 baseline	generic 3-model blend, raw 8 features	0.56166
exp 2	engineered 24-feature LGB/XGB/CAT weight-searched blend (LGB .45 / XGB .15 / CAT .40)	0.55877
exp 4 (round 1)	+ Optuna-tuned LGB as 4th member	0.557977
exp 5 (round 2)	+ seed-bagged tuned LGB (seed 2024) as 5th member	0.557859
exp 7 (round 4)	5-member pool retrained on 26 features (+knn/coastal geo)	0.557088
exp 8 (tree-search node #14)	7-member clip-aware blend + ceiling-classifier hybrid	0.556329

The **Loss Function** is squared error — LightGBM `rmse / regression` objective, XGBoost `reg:squarederror`, CatBoost `RMSE` loss — with model selection directly on OOF RMSE. The **Optimization Algorithm** is gradient boosting (histogram-based) — **not** SGD/ADAM — with blend weights optimized by grid search (linear rounds) and Dirichlet + coordinate-ascent (tree search). The **Learning Rate** is the boosting shrinkage (base LGB/XGB/CAT `learning_rate` = 0.03; the Optuna-tuned LGB uses ≈ 0.0116); there is **no Learning Rate Scheduler** (N/A — *GBDT convergence is governed by CV early-stopping over boosting rounds, not a schedule*) and **no Batch Size** (N/A — *full-dataset histogram boosting, not mini-batched*).

Training Duration for the tree-search champion was **~ 190 s of wall-clock over 22 evaluated nodes (22/22 model fits completed; 23 total incl. 1 cleanly-failed node)**; the earlier linear pipeline was cheaper still (3-model $\times$ 5-fold base fit ≈ 45.5 s; Optuna tune 50 trials / 74.3 s; round-4 full retrain 67.2 s) — every stage well within the compute budget. **Training Memory Consumption Limits** were not explicitly capped — a $37k \times 26$ float matrix is trivial on the single multi-core CPU machine, and peak memory was not separately measured (**not recorded**). One engineering note worth keeping: explicit `n_jobs=-1` on the LGB/XGB wrappers caused a 10–50 $\times$ slowdown from core oversubscription on this sandbox; leaving `n_jobs` at default restored 1.6–2 s/fold.

Transfer Learning is *N/A (no pretrained weights)* — the rules forbid pretrained models anyway; its analogue here is **cross-competition experience injection**, where the tree search drew priors from an experience library (`suggest_priors` returned P0–P19); prior-informed ensemble/tuning mutations won 5/5, while the two biggest gains came from comp-local top-code EDA insight absent from the library. **Data Augmentation** is *N/A (tabular)*; the analogues are feature engineering (the 26 features) and **seed-bagging** (a second tuned-LGB at seed 2024, later seed 777).

Reproducibility Standards are strict: fixed seeds (KFold seed = 42; Optuna and seed-bag seeds logged), LightGBM determinism care, and a byte-identical processed-CSV artifact used because LightGBM proved chaotically sensitive to $\sim 1e-13$ float differences at `num_leaves` = 121 (a 0.00026 OOF shift from sub-ULP recomputation noise). All 5 linear pool members were reproduced digit-for-digit before searching, and the **faithful replay reproduces the champion 22/22 exactly** ($\Delta = -2e-6$).

Inference procedures

Decision Threshold is *N/A* — RMSE is a continuous regression metric, so we submit raw real-valued predictions and never threshold. The one relevant post-process is `clip_to_train_target_range`: predictions are clipped to the observed train target span (notably the 5.00001 top-code); the tree search additionally applied this clip *inside* the OOF metric during weight search (not just at submission time), which was worth +0.000123 at the 7-way layer — free accuracy the linear run had left on the table. **Inference Duration** was not separately profiled (**not recorded**); GBDT scoring of the 24,759 test rows is sub-second per member on CPU. **Inference Memory Consumption Limits** are negligible (tree inference is lightweight, not capped; **not recorded**).

Evaluation & Benchmarking

Task for Performance Evaluation. Predict the median house value for every test census block. **Performance Metrics.** RMSE (minimize) is the sole competition metric; our tree-search champion reaches **CV OOF RMSE 0.556329**, improving $0.56166 \rightarrow 0.55877 \rightarrow 0.557088 \rightarrow 0.556329$ across baseline, engineered blend, linear-iteration best, and tree search ($\approx 0.81\%$ relative reduction overall from the baseline; the tree search added a further -0.000759 vs the linear best, $\approx 0.14\%$ relative). No public/private leaderboard number exists for this run — it was CV-only.

Performance Benchmarking. The natural comparator is the **NVIDIA reproduce-agent**, which copies the strongest public kernel verbatim:

Agent	Approach	OOF RMSE	Note
NVIDIA	reproduces <code>dmitryuarov/ps-s3e1-coordinates-key-to-victory</code> (verbatim)	0.5086	winner
Our agent	from-scratch GBDT pool + tree-search v2	0.556329	behind

NVIDIA leads clearly here (0.5086 vs 0.5563). The gap is not a modelling-mechanics gap but a feature-idea gap: the public kernel's headline is built on aggressive **coordinate feature engineering** ("coordinates — key to victory"), a comp-specific geo-encoding recipe that our raw-Lat/Long + distance-to-city + KNN features

only partially recover. Our own run confirms the same lesson from the inside — its two biggest wins (clip-aware OOF scoring and the ceiling-classifier hybrid) came from local top-code EDA, while transferred ensemble priors delivered only reliable-but-small gains. On this S3-era geo-regression problem, a well-optimized public coordinates kernel retains a substantial edge; the reproduce-vs-originate gap narrows to dead heats on several other seasons but not on s3e1.